

Кафедра комп'ютерних
систем і мереж

ЛАБОРАТОРНА РОБОТА №9
з дисципліни **Теорія алгоритмів і структури даних**
Тема:
Способи організації списків

Виконала:
ст. гр. КІ-25-1
Лаврів А.
Перевірив:
Гарасимів Т.Г.

м. Івано-Франківськ

2026

Мета: ознайомитися із основними способами організації списків та особливостями їх програмної реалізації; набути практичних навичок по роботі з простими списками двох типів: на основі масивів та на основі вказівників.

Завдання:

1. Реалізувати аналогічну програму для роботи зі своїм варіантом структури з реалізацією наступних функцій:

- очистити список;
- додати елемент до списку (в кінець) ;
- вилучити вказаний елемент зі списку; –визначити кількість елементів у списку;
- поміняти два сусідні елементи місцями, які слідує за позицією p ;
- об’єднати два списки в один (з утворенням нового списку або шляхом дописування другого списку в кінець першого списку);
- одержати доступ до k -го вузла списку;
- включити новий вузол безпосередньо перед k -им вузлом; –розбити лінійний список на два (або більше) списки;
- зробити копію лінійного списку;
- знайти в списку вузол із заданим значенням у деякій полі. При виконанні всіх дій передбачити відображення поточного стану списку.

Код:

```
#include <iostream>
#include <string>
#include <iomanip>
```

```
using namespace std;
```

```
struct Film
{
    int code;
    string title;
    int year;
    string studio;
};
```

```
struct Node
{
    Film data;
    Node* prev;
    Node* next;
};
```

```
class DoublyList
{
private:
    Node* head;
    Node* tail;
```

```
public:
    DoublyList()
    {
        head = NULL;
        tail = NULL;
    }
```

```
void addFilm(Film film)
{
    Node* p = new Node;

    p->data = film;
    p->next = NULL;
    p->prev = tail;

    if (head == NULL)
    {
        head = p;
        tail = p;
    }
    else
    {
        tail->next = p;
        tail = p;
    }
}
```

```

    cout << "Додано фільм: " << film.title << endl;
}

// Виведення списку таблицею
void printList()
{
    Node* p = head;

    if (p == NULL)
    {
        cout << "Список порожній!" << endl;
        return;
    }

    cout << endl;
    cout <<
    "=====
=====
" << endl;

    cout << left
        << setw(10) << "Код"
        << setw(25) << "Назва"
        << setw(15) << "Рік"
        << setw(30) << "Студія" << endl;

    cout <<
    "=====
=====
" << endl;

    while (p != NULL)
    {
        cout << left
            << setw(10) << p->data.code
            << setw(25) << p->data.title
            << setw(15) << p->data.year
            << setw(30) << p->data.studio
            << endl;

        p = p->next;
    }
}

```

```

        cout <<
"=====
===== " << endl;
    }

// Кількість елементів
int countElements()
{
    int count = 0;
    Node* p = head;

    while (p != NULL)
    {
        count++;
        p = p->next;
    }

    return count;
}

// Пошук фільму
void findFilm(string name)
{
    Node* p = head;

    while (p != NULL)
    {
        if (p->data.title == name)
        {
            cout << endl;
            cout << "Фільм знайдено!" << endl;
            cout << "Назва: " << p->data.title << endl;
            cout << "Рік: " << p->data.year << endl;
            cout << "Студія: " << p->data.studio << endl;
            return;
        }

        p = p->next;
    }
}

```

```

    cout << "Фільм не знайдено!" << endl;
}

// Видалення фільму
void deleteFilm(int code)
{
    Node* p = head;

    while (p != NULL)
    {
        if (p->data.code == code)
        {
            if (p == head)
                head = p->next;

            if (p == tail)
                tail = p->prev;

            if (p->prev != NULL)
                p->prev->next = p->next;

            if (p->next != NULL)
                p->next->prev = p->prev;

            cout << endl;
            cout << "Видалено фільм: " << p->data.title << endl;

            delete p;
            return;
        }

        p = p->next;
    }

    cout << "Фільм не знайдено!" << endl;
}

```

```

void swapNeighbors()
{

```

```
if (head == NULL || head->next == NULL)
{
    cout << "Недостатньо елементів для перестановки!" << endl;
    return;
}
```

```
Film temp = head->data;
head->data = head->next->data;
head->next->data = temp;
```

```
cout << endl;
cout << "Перші два елементи поміняно місцями!" << endl;
}
```

// Очищення списку

```
void clearList()
```

```
{
    Node* p = head;

    while (p != NULL)
    {
        Node* temp = p;
        p = p->next;
        delete temp;
    }
```

```
head = NULL;
tail = NULL;
```

```
cout << endl;
cout << "Список очищено!" << endl;
```

```
}
```

```
};
```

```
int main()
```

```
{
    DoublyList films;
```

```
Film f1 = {101, "Avatar", 2009, "20th Century Fox"};
Film f2 = {102, "Titanic", 1997, "Paramount Pictures"};
```

```
Film f3 = {103, "Inception", 2010, "Warner Bros"};  
Film f4 = {104, "Interstellar", 2014, "Paramount Pictures"};  
Film f5 = {105, "Joker", 2019, "Warner Bros"};  
Film f6 = {106, "The Matrix", 1999, "Warner Bros"};  
Film f7 = {107, "Gladiator", 2000, "DreamWorks"};  
Film f8 = {108, "Oppenheimer", 2023, "Universal Pictures"};
```

```
cout << "===== ДОДАВАННЯ ФІЛЬМІВ =====" << endl;
```

```
films.addFilm(f1);  
films.addFilm(f2);  
films.addFilm(f3);  
films.addFilm(f4);  
films.addFilm(f5);  
films.addFilm(f6);  
films.addFilm(f7);  
films.addFilm(f8);
```

```
cout << endl;
```

```
cout << "===== ПОЧАТКОВИЙ СПИСОК =====" << endl;
```

```
films.printList();
```

```
cout << endl;
```

```
cout << "Кількість елементів у списку: ";
```

```
cout << films.countElements() << endl;
```

```
cout << endl;
```

```
cout << "===== ПОШУК ФІЛЬМУ =====" << endl;
```

```
films.findFilm("Interstellar");
```

```
cout << endl;
```

```
cout << "===== ПЕРЕСТАНОВКА =====" << endl;
```

```
films.swapNeighbors();
```

```
cout << endl;
```

```
cout << "Список після перестановки:" << endl;
```



```

films.printList();

cout << endl;
cout << "===== ВИДАЛЕННЯ =====" << endl;

films.deleteFilm(105);

cout << endl;
cout << "Список після видалення:" << endl;

films.printList();

cout << endl;
cout << "===== ОЧИЩЕННЯ СПИСКУ =====" << endl;

films.clearList();

cout << endl;
cout << "Стан списку після очищення:" << endl;

films.printList();

return 0;
}

```

```

===== ДОДАВАННЯ ФІЛЬМІВ =====
Додано фільм: Avatar
Додано фільм: Titanic
Додано фільм: Inception
Додано фільм: Interstellar
Додано фільм: Joker
Додано фільм: The Matrix
Додано фільм: Gladiator
Додано фільм: Oppenheimer

===== ПОЧАТКОВИЙ СПИСОК =====

```

Код	Назва	Рік	Студія
101	Avatar	2009	20th Century Fox
102	Titanic	1997	Paramount Pictures
103	Inception	2010	Warner Bros
104	Interstellar	2014	Paramount Pictures
105	Joker	2019	Warner Bros
106	The Matrix	1999	Warner Bros
107	Gladiator	2000	DreamWorks
108	Oppenheimer	2023	Universal Pictures

```

Кількість елементів у списку: 8

===== ПОШУК ФІЛЬМУ =====
Фільм знайдено!
Назва: Interstellar
Рік: 2014
Студія: Paramount Pictures

```

Рис. 1 - Демонстрація роботи коду

```
===== ПЕРЕСТАНОВКА =====  
  
Перші два елементи поміняно місцями!  
  
Список після перестановки:  
  
=====
```

Код	Назва	Рік	Студія
102	Titanic	1997	Paramount Pictures
101	Avatar	2009	20th Century Fox
103	Inception	2010	Warner Bros
104	Interstellar	2014	Paramount Pictures
105	Joker	2019	Warner Bros
106	The Matrix	1999	Warner Bros
107	Gladiator	2000	DreamWorks
108	Oppenheimer	2023	Universal Pictures

```
=====
```

Рис.2 - Демонстрація роботи коду

```
===== ВИДАЛЕННЯ =====  
  
Видалено фільм: Joker  
  
Список після видалення:  
  
=====
```

Код	Назва	Рік	Студія
102	Titanic	1997	Paramount Pictures
101	Avatar	2009	20th Century Fox
103	Inception	2010	Warner Bros
104	Interstellar	2014	Paramount Pictures
106	The Matrix	1999	Warner Bros
107	Gladiator	2000	DreamWorks
108	Oppenheimer	2023	Universal Pictures

```
=====
```

```
===== ОЧИЩЕННЯ СПИСКУ =====  
  
Список очищено!  
  
Стан списку після очищення:  
Список порожній!  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

Рис. 3 - Демонстрація роботи коду

Висновок: У ході виконання лабораторної роботи було реалізовано двозв'язний список та основні операції роботи з ним. Було закріплено навички роботи з динамічною пам'яттю, вказівниками та структурами даних. Також було досліджено особливості додавання, видалення, пошуку та перестановки елементів у списках.